I ended up going down to 5 arms.

1. When using the (warmup + selection) samples for estimation, the mean 95% credible interval width is 55% wider under equal randomization compared to adaptive randomization.

2. I was not yet able to show bias in the selection set

3. When using the (validation) sample for estimation, the two approaches show disappointingly similar performance.

Simulation for Adaptive Conjoint Appendix

```
library(tidyverse)
theme_set(theme_bw())
library(foreach)
library(doParallel)
library(doRNG)
cl <- makeCluster(detectCores())
registerDoParallel(cl)
```

Define true parameter to learn.

```
theta <- tibble(
  arm = 1:5,
  theta = seq(.3,.7,.1)
)
```

Define an estimator that samples arms with equal probabilities that are counterbalanced to have equal sample sizes.

```
equal_probability_estimator <- function(total_n, arms = theta$arm) {
  if (length(arms) == 1) {
    data_aggregated <- tibble(id = 1:total_n) |>
      mutate(
        arm = arms
      )
  } else {
    data_aggregated <- tibble(id = 1:total_n) |>
      mutate(
        arm = replicate(
          n = ceiling(total_n / length(arms)),
          expr = sample(arms)
        )[1:total_n]
```

```
    )
  }
  data_aggregated <- data_aggregated |>
    # Merge in true choice probabilities
    left_join(theta, by = join_by(arm)) |>
    # Simulate the choice outcome
    mutate(y = rbinom(n(), 1, theta)) |>
    # Count choice outcomes by arm
    group_by(arm) |>
    summarize(y1 = sum(y == 1), y0 = sum(y == 0))

  # Simulate from posterior beta distribution
  estimates <- foreach(sim_rep = 1:1e3, .combine = "rbind") %do% {
    data_aggregated |>
      mutate(
        # Simulate theta from the posterior
        theta = rbeta(n(), 1 + y1, 1 + y0)
      ) |>
      # Determine which theta is biggest in this sim_rep
      arrange(theta) |>
      mutate(is_extreme = 1:n() == n())
  } |>
    select(arm, theta, is_extreme) |>
    group_by(arm) |>
    summarize(
      theta_hat = mean(theta),
      theta_ci_min = quantile(theta, .025),
      theta_ci_max = quantile(theta, .975),
      p_max = mean(is_extreme)
    )

  return(estimates)
}
```

Define an adaptive estimator with `warmup_n` cases used for warmup and `adaptive_n` cases used for adaptive estimation.

```
adaptive_estimator <- function(warmup_n, adaptive_n, arms = theta$arm) {

  # Warm-up phase
  data_aggregated <- tibble(id = 1:warmup_n) |>
    # Assign arms systematically with equal probabilities
```

```
  mutate(
    arm = replicate(
      n = ceiling(warmup_n / length(arms)),
      expr = sample(arms)
    )[1:warmup_n]
  ) |>
  # Merge in true parameter values
  left_join(theta, by = join_by(arm)) |>
  # Simulate data
  mutate(y = rbinom(n(), 1, theta)) |>
  group_by(arm) |>
  summarize(y1 = sum(y), y0 = sum(1 - y)) |>
  select(arm, y1, y0)

for (i in 1:adaptive_n) {
  # Adaptively update 1 arm with 1 new data point
  updated_arm <- data_aggregated |>
    # Simulate 1 posterior theta value for each arm
    mutate(sim_theta = rbeta(n(), 1 + y1, 1 + y0)) |>
    # Keep the largest theta value
    arrange(sim_theta) |>
    slice_tail(n = 1) |>
    # Draw data from that arm, using true (unknown) theta
    left_join(theta, by = join_by(arm)) |>
    mutate(y = rbinom(1, 1, theta)) |>
    # Update aggregate data
    mutate(
      y1 = y1 + y,
      y0 = y0 + (1 - y)
    ) |>
    select(arm, y1, y0)

  # Update the aggregated data for that arm
  data_aggregated <- data_aggregated |>
    filter(arm != updated_arm$arm) |>
    bind_rows(updated_arm)
}

# Simulate from posterior beta distribution
estimates <- foreach(sim_rep = 1:1e3, .combine = "rbind") %do% {
  data_aggregated |>
    mutate(
```

```
        # Simulate a value of theta from the posterior
        theta = rbeta(n(), 1 + y1, 1 + y0)
      )  |>
      # Determine which theta is biggest in this sim_rep
      arrange(theta) |>
      mutate(is_extreme = 1:n() == n())
  } |>
    select(arm, theta, is_extreme) |>
    group_by(arm) |>
    summarize(
      theta_hat = mean(theta),
      theta_ci_min = quantile(theta, .025),
      theta_ci_max = quantile(theta, .975),
      p_max = mean(is_extreme)
    )

  return(estimates)
}
```

Write a function that calls the above to make all 2×2 cells of estimates that invovle:

1) One-sample estimation vs
2) Selection followed by a separate validation set

and

1) Equal probability randomization for selection
2) Warmup + adaptive randomization for selection

Throughout, the total sample size is kept constant. As a result, an estimator that does not involve a selection step gets to use more observations for the single-sample estimation.

```
simulate_selection_bias <- function(total_n, selection_n, warmup_n) {

  # Define sample sizes implicit in arguments
  validation_n <- total_n - selection_n
  adaptive_n <- selection_n - warmup_n

  # Carry out equal and adaptive estimation in the full sample (no split)
  equal_full_sample <- equal_probability_estimator(total_n = total_n)
  adaptive_full_sample <- adaptive_estimator(
    warmup_n = warmup_n,
    adaptive_n = total_n - warmup_n,
```

```
  arms = theta$arm
)

# Carry out selection estimation in a selection sample
equal_selection_sample <- equal_probability_estimator(total_n = selection_n)
adaptive_selection_sample <- adaptive_estimator(
  warmup_n = warmup_n,
  adaptive_n = adaptive_n,
  arms = theta$arm
)

# Determine which arm had the max probability of being the max in each estimator
equal_selected <- equal_selection_sample |> arrange(p_max) |> slice_tail(n = 1) |> pull(arm
adaptive_selected <- adaptive_selection_sample |> arrange(p_max) |> slice_tail(n = 1) |> pu

# Carry out validation estimation for selected cases
equal_validation <- equal_probability_estimator(
  total_n = validation_n,
  arms = equal_selected
)
adaptive_validation <- equal_probability_estimator(
  total_n = validation_n,
  arms = adaptive_selected
)

return(
  equal_full_sample |> mutate(sample = "full", randomization = "equal")|>
    bind_rows(
      adaptive_full_sample |> mutate(sample = "full", randomization = "adaptive")
    ) |>
    bind_rows(
      equal_selection_sample |> mutate(sample = "selection", randomization = "equal")
    ) |>
    bind_rows(
      adaptive_selection_sample |> mutate(sample = "selection", randomization = "adaptive"
    ) |>
    bind_rows(
      equal_validation |> mutate(sample = "validation", randomization = "equal")
    )  |>
    bind_rows(
      adaptive_validation |> mutate(sample = "validation", randomization = "adaptive")
    )
```

```
  )
}
```

The code below applies the functions to carry out the simulation.

```
t0 <- Sys.time()
selection_simulations <- foreach(
  rep = 1:1e3,
  .combine = "rbind",
  .packages = c("tidyverse","foreach")
) %dorng% {
  simulate_selection_bias(
    total_n = 500, selection_n = 250, warmup_n = 50
    # bottom is slightly better for showing bias of selection set,
    # but main claims get noisier and less clear
    #total_n = 300, selection_n = 200, warmup_n = 50
  ) |>
    mutate(simulation = rep)
}
spent <- difftime(Sys.time(),t0)
```

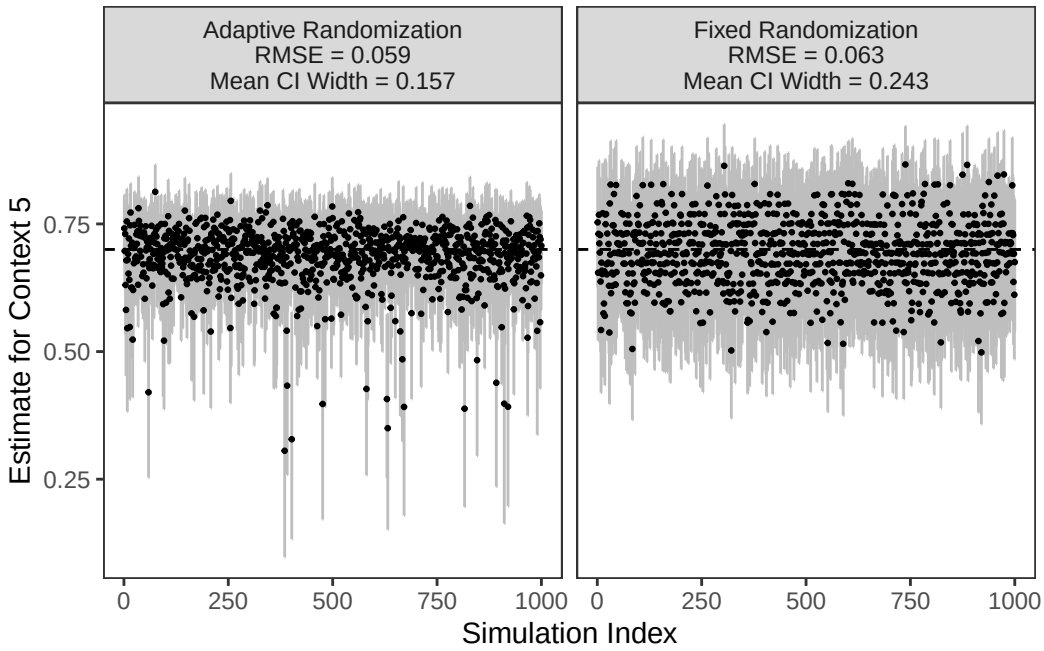
Visualize adaptive vs equal randomization estimation in the selection set.

```
forplot <- selection_simulations |>
  filter(arm == 5) |>
  filter(sample == "selection") |>
  left_join(theta, by = join_by(arm))

summaries <- forplot |>
  group_by(randomization) |>
  summarize(
    mean_interval_width = mean(theta_ci_max - theta_ci_min),
    rmse = sqrt(mean((theta_hat - .7) ^ 2)),
    bias = mean(theta_hat - .7)
  )

summaries |>
  pivot_longer(cols = -randomization) |>
  pivot_wider(names_from = "randomization") |>
  mutate(equal_over_adaptive = equal / adaptive)
```

```
# A tibble: 3 x 4
  name                adaptive    equal equal_over_adaptive
  <chr>                  <dbl>    <dbl>               <dbl>
1 mean_interval_width   0.157   0.243                1.55
2 rmse                  0.0594  0.0632               1.06
3 bias                 -0.0144 -0.00804              0.559
```

```
forplot |>
  left_join(summaries, by = join_by(randomization)) |>
  mutate(
    randomization = case_when(
      randomization == "equal" ~ "Fixed Randomization",
      randomization == "adaptive" ~ "Adaptive Randomization"
    ),
    randomization = paste0(
      randomization,"\nRMSE = ",
      scales::label_number(accuracy = .001)(rmse),
      "\nMean CI Width = ",
      scales::label_number(accuracy = .001)(mean_interval_width)
    )
  ) |>
  ggplot(aes(
    y = theta_hat,
    x = simulation,
    ymin = theta_ci_min,
    ymax = theta_ci_max
  )) +
  geom_errorbar(width = 0, color = "gray") +
  geom_point(size = .5) +
  geom_hline(
    yintercept = theta |> filter(arm == 5) |> pull(theta),
    linetype = "dashed"
  ) +
  facet_wrap(~randomization, nrow = 1) +
  xlab("Simulation Index") +
  ylab("Estimate for Context 5") +
  theme(panel.grid = element_blank())
```

Point (1) from summary is this figure.
1. When using the (warmup + selection) samples for estimation, the mean 95% credible interval width is 55% wider under equal randomization compared to adaptive randomization.



With adaptive estimation, on average the posterior probability that Arm 5 is the correct arm is higher.

```
selection_simulations |>
  filter(sample == "selection") |>
  filter(arm == 5) |>
  group_by(randomization) |>
  summarize(p_max = mean(p_max))
```

```
# A tibble: 2 x 2
  randomization p_max
  <chr>         <dbl>
1 adaptive      0.789
2 equal         0.730
```

This means you choose to focus on the correct arm at a slightly higher rate at a given sample size.

```
selection_estimates <- selection_simulations |>
  filter(sample == "selection") |>
  # Keep only the arm that would be selected as the max
  group_by(randomization, simulation) |>
  filter(p_max == max(p_max))
```

```
selection_estimates |>
  # Summarize how often that is correct
  group_by(randomization) |>
  summarize(correct_arm = mean(arm == 5))
```

```
# A tibble: 2 x 2
  randomization correct_arm
  <chr>                <dbl>
1 adaptive             0.904
2 equal                0.838
```

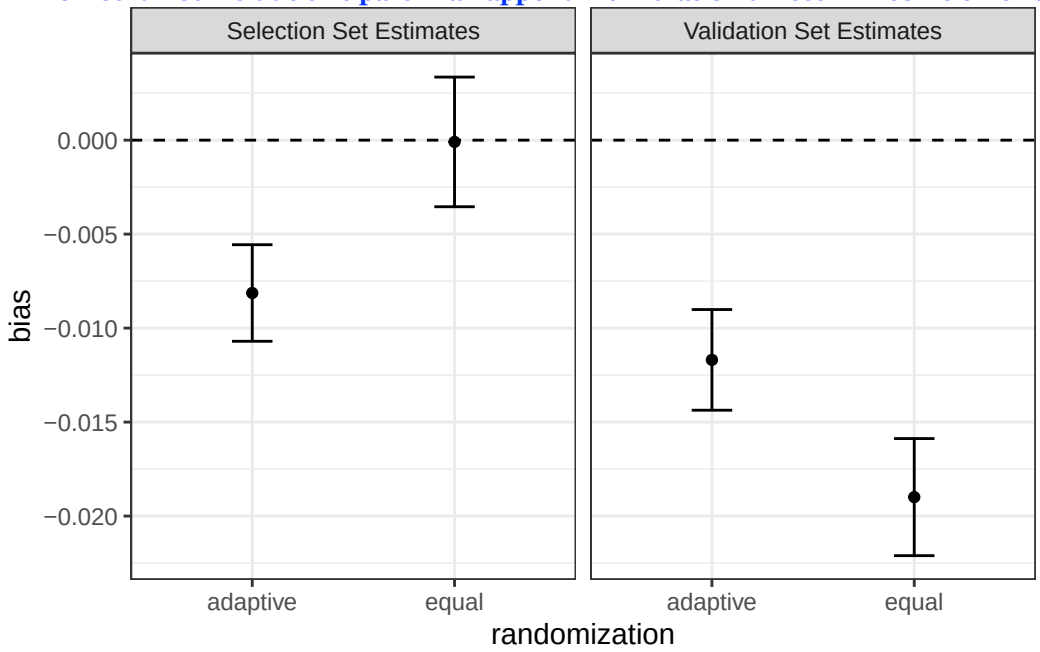
Relevant to point (2) below, this output shows that the adaptive randomization chose the correct arm 90% of the time compared to 83.8% of the time for fixed randomization.

There is a potential problem of bias induced by selecting the highest-estimate arm. This can be solved with a validation set, but at a cost to efficiency.

```
bias_selection <- selection_estimates |>
  group_by(randomization) |>
  summarize(
    bias = mean(theta_hat - .7),
    bias_se = sd(theta_hat) / sqrt(n())
  ) |>
  mutate(sample = "Selection Set Estimates")
bias_validation <- selection_simulations |>
  filter(sample == "validation") |>
  group_by(randomization) |>
  summarize(
    bias = mean(theta_hat - .7),
    bias_se = sd(theta_hat) / sqrt(n())
  ) |>
  mutate(sample = "Validation Set Estimates")
bias_selection |>
  bind_rows(bias_validation) |>
  ggplot(aes(
    x = randomization,
    y = bias,
    ymin = bias - qnorm(.975) * bias_se,
    ymax = bias + qnorm(.975) * bias_se
  )) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  geom_point() +
  geom_errorbar(width = .2) +
  facet_wrap(~sample)
```

(Point 2 from summary is this figure)
2. I was not yet able to show bias in the selection set

I think this happens because
- both approaches choose the correct arm 80%+ of the time
- all estimates are a tiny bit downward biased because I am using Bayes posterior estimates anyhow
Overall, I think we would not include this part in an appendix simulation unless I fix some error so that it is more compelling.



Visualize adaptive vs equal randomization estimation in the validation set.

```
forplot <- selection_simulations |>
  filter(sample == "validation")

summaries <- forplot |>
  group_by(randomization) |>
  summarize(
    mean_interval_width = mean(theta_ci_max - theta_ci_min),
    rmse = sqrt(mean((theta_hat - .7) ^ 2)),
    bias = mean(theta_hat - .7)
  )

summaries |>
  pivot_longer(cols = -randomization) |>
  pivot_wider(names_from = "randomization") |>
  mutate(equal_over_adaptive = equal / adaptive)
```

```
# A tibble: 3 x 4
  name                adaptive   equal equal_over_adaptive
  <chr>                  <dbl>   <dbl>               <dbl>
1 mean_interval_width   0.113   0.113                1.00
2 rmse                  0.0447  0.0537               1.20
3 bias                 -0.0117 -0.0190               1.62
```

```r
forplot |>
  left_join(summaries, by = join_by(randomization)) |>
  mutate(
    randomization = case_when(
      randomization == "equal" ~ "Fixed Randomization",
      randomization == "adaptive" ~ "Adaptive Randomization"
    ),
    randomization = paste0(
      randomization,"\nRMSE = ",
      scales::label_number(accuracy = .001)(rmse),
      "\nMean CI Width = ",
      scales::label_number(accuracy = .001)(mean_interval_width)
    )
  ) |>
  ggplot(aes(
    y = theta_hat,
    x = simulation,
    ymin = theta_ci_min,
    ymax = theta_ci_max
  )) +
  geom_errorbar(width = 0, color = "gray") +
  geom_point(size = .5) +
  geom_hline(
    yintercept = theta |> filter(arm == 5) |> pull(theta),
    linetype = "dashed"
  ) +
  facet_wrap(~randomization, nrow = 1) +
  xlab("Simulation Index") +
  ylab("Estimated Maximum Theta") +
  theme(panel.grid = element_blank())
```

(Point 3 from summary is this figure)
3. When using the (validation) sample for estimation, the two approaches show disappointingly similar performance.